Reproducing SRSNet: a closer look at Selective Patching and Adaptive Fusion

Andrea Perugini *peruga@usi.ch*
Dominik Panzarella *panzad@usi.ch*
Susanna Ardigò *ardigs@usi.ch*

Abstract

In this work, we reproduced the ETT-facing experiments of SRSNet using the released code and methods. Since the full benchmark was too expensive on our available hardware and some released scripts were incomplete, we restricted the reproduction to the four ETT datasets and four forecast horizons. SRSNet remains close to the strongest baselines, particularly on MAE, but the numerical results differ from the paper and fail to display the clear dominance reported by the authors. As a follow-up, we tested several controls for the SRS mechanism, replacing learned selection with random or engineered alternatives, and we also introduced a context-dependent hypernetwork in lieu of the fixed fusion weight. Our extensions remain within the observed seed-level variation, thereby suggesting that, at least insofar as our reproduction scope is concerned, we cannot clearly isolate learned Selective Patching as the main source of SRSNet's performance.

1 Background and model

Patching in time series forecasting. Long-term time series forecasting predicts the next L values given a lookback window of past observations $X \in \mathbb{R}^{N \times T}$ with N channels and T timesteps. Recent work has converged on a simple recipe inspired by Vision Transformers: cut the lookback into short *patches*, embed each patch into a vector, run a backbone over the patch tokens, and read the forecast from a small head. PatchTST (Nie et al., 2023), Crossformer (Zhang & Yan, 2023) and PatchMLP (Tang & Zhang, 2025) all share this template, channel- independent strategy included. Patches make the input more semantically rich than a single timestep, and reduce the sequence length the backbone has to deal with.

The problem the paper raises. All these models cut the lookback into *adjacent* patches with fixed stride s. The slicing is the same for every input and every dataset. Wu et al. (2025) argue that this is limiting on three fronts: a fixed grid can split distribution- shift boundaries (changes of regime, anomalies, shifts) in the middle of a patch; quasi-periodic signals produce consecutive patches that are near-copies of each other; and the choice does not adapt to the data. The paper's claim is that a more flexible patching, picked from the input itself, should help.

The SRS module. The proposed module, called the *Selective Representation Space* (SRS), builds two views of the same window and blends them. The first view is the conventional adjacent patching already used by every other patch-based model. The second view, the *selective view*, is constructed in two steps.

Selective Patching (SP) considers every possible sub-window of length p at stride 1 (so $K = (n-1)\,s + 1$ candidates with $T = 96$, $p = 24$ and $s = 24$, this gives 73 candidates) and scores each candidate with a small MLP, the *Scorer*^{s}. Argmax over the score axis then selects the top n candidates.

Dynamic Reassembly feeds the n selected patches into a second MLP, the *Scorer*^{r}, which scores each one. Argsort then turns these scores into an ordering: for example, scores $[0.3, 0.9, 0.1]$ would place the second patch first.

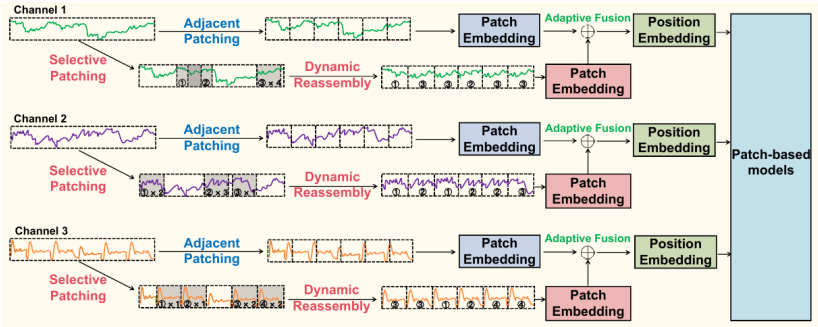

Figure 1: SRSNet pipeline (reproduction of Wu et al. 2025, Fig. 2). The adjacent and selective streams are linearly embedded, blended by the Adaptive Fusion through α, summed with a sinusoidal positional embedding, and passed to the linear head.

Because both Argmax and Argsort are non-differentiable, the paper uses a straight-through trick to propagate gradients back to the two scorers: each selected (or reordered) patch is multiplied by a tensor that is numerically equal to 1 but carries the gradient of the scorer's output. This is what allows the scorers to be trained end-to-end with the rest of the model.

Both views, after their own linear patch-embedding, become two tensors $E^c, E^s \in \mathbb{R}^{N\times n\times d}$. The *Adaptive Fusion* (AF) merges them with a learned weight $\alpha \in [0,1]^{n\times d}$:

$$\tilde{E} \;=\; \alpha \odot E^c + (1-\alpha)\odot E^s.$$

In practice α is a free `nn.Parameter` of shape $[n, d_{\text{model}}]$ wrapped in a sigmoid so the constraint $\alpha \in [0,1]$ is satisfied automatically. It is learned during training but does not depend on the input at inference time: the same α is applied to every forward pass once training is done. The paper itself flags this in its future work section as a place where a more adaptive mechanism could help. A sinusoidal positional embedding is finally added on top.

SRSNet. The released implementation uses RevIN (Kim et al., 2022), the SRS module, and a FlattenHead. Figure 1 shows the full pipeline reproduced from the paper.

This makes SRSNet a useful reproducibility target for two reasons. First, the paper reports strong benchmark results against several forecasting models. Second, we can actually take apart and test its claimed mechanism, i.e., if learned SP is central to the model, then randomising or removing altogether the selector should hurt performance. Our report can therefore be divided into two parts: we first reproduced the ETT-facing experiments across the headline comparison, the plug-in study, and ablation; we then ran extensions targeting the learned selection, the fusion weight, and the forecasting head.

2 What we wanted to test

The paper makes two claims that are relevant to our project: first, that the SRS module improves over a simple MLP forecaster; second, that the individual SRS components (namely, Selective Patching, Dynamic Reassembly, and Adaptive Fusion) are important, with their ablation study attributing the greatest impact to Selective Patching. Thus, on top of reproducing these tests, we wanted to test whether the learned selector itself was necessary as a complementary control **RandomSP**. We kept the selective-patching structure, but replaced the learned top-k scoring with random selection. Therefore, if learned patch selection is responsible for the observed gain, using random choices should hurt consistently. As a stricter version of this control, **RandomSPNoShuffle** additionally disables the learned Scorer^r (Dynamic Reassembly), so the random patches are kept in the order in which they were sampled. This way, we also probe whether the learned reordering carries any independent value beyond the selection itself.

After that destructive control, we also introduced two other constructive variants in our extensions. Time-Aware Selective Patching (**TASP**) swaps the raw patch scoring input with four simple time-series descriptors, so we explore whether patch selection can be made both more interpretable and more efficient. In practice, the engineered features that we chose are dominant FFT magnitude, lag-1 autocorrelation, variance, and trend slope. Next, Hypernet Adaptive Fusion (**HypernetAF**) instead does not target the selector, but it replaces the static fusion parameter α with a small per-batch context-dependent two-layer network. More explicitly, in vanilla SRSNet the adjacent-patch embedding E^c and the selective/reassembled embedding E^s are combined using a learned tensor $\alpha \in [0,1]^{n\times d}$ (in practice a free `nn.Parameter` wrapped in a sigmoid of shape $[n, d_{\text{model}}]$ like in the standard AF explained above):

$$\tilde{E} = \alpha \odot E^c + (1-\alpha) \odot E^s.$$

This tensor is learned during training, but at inference the same fusion weights are used for every input. Conversely, in HypernetAF, after computing the adjacent-view embedding E^c, we average it over the batch/channel axis and over patches to obtain a context vector $c \in \mathbb{R}^d$. This context is passed through a small two-layer network whose output is reshaped and wrapped in the same sigmoid as vanilla AF:

$$h = \text{ReLU}(W_1 c + b_1), \qquad \alpha_{\text{dyn}} = \text{sigmoid}\big(\text{reshape}(W_2 h + b_2)\big),$$

with $\alpha_{\text{dyn}} \in [0,1]^{n\times d}$. The output bias b_2 is initialised to 3.5, so at step 0 every component of α_{dyn} is $\sigma(3.5) \approx 0.97$, matching the vanilla initialisation used by the authors in the ETTh1 scripts. This extension was further motivated by the fact that Adaptive Fusion emerged as the most influential component in our reproduced ablation. If fusion is indeed important, we might improve robustness by letting the fusion weight depend on the batch context.

Lastly, to check whether the simple linear forecasting head is a bottleneck, we tested the inclusion of a small Transformer encoder following the SRS embedding. In practice, the encoder is a standard `nn.TransformerEncoder` with two layers, four attention heads, and a feedforward dimension equal to $4d_{\text{model}}$, inserted between the SRS module and the FlattenHead. To probe whether the encoder interacts with the selection or fusion side, we re-ran three variants in combination with it: vanilla SRSNet, TASP, and HypernetAF.

3 Reproducibility

We ran the experiments in the paper through the official TFB pipeline (Qiu et al., 2024) with paper-faithful overrides (`batch_size=64`, `train_drop_last=false`, `num_workers=0`). All runs were on a single RTX 4090 (24 GB) on Hivenet. A part of the runs was also double-tested locally on the same GPU with comparable outcomes (results not reported here).

The paper covers eight benchmark datasets (the four ETT, plus Weather, Solar, Traffic, Electricity). Running the full grid on our hardware would have proven very time-consuming, as a single SRSNet run on Traffic or Electricity at horizon 720 takes several wall-clock hours with a batch size of 64. Thus, we narrowed down our scope to the four ETT datasets (ETTh1, ETTh2, ETTm1, and ETTm2) and the four standard horizons (96, 192, 336, 720). Within this scope, the headline comparison covered SRSNet and the main released baselines used in the paper: PatchTST, DLinear, iTransformer, TimesNet, TimeMixer, and PatchMLP. Coverage was still not uniform: the released ETTh1 scripts for DLinear, iTransformer, TimesNet, and TimeMixer are empty, while PatchMLP scripts are absent for ETTh2 and ETTm1. We therefore report missing cells explicitly rather than filling them with non-paper-faithful runs.

3.1 Headline comparison (paper Tab. 2)

Table 1 averages MSE and MAE over the four horizons for each (model, dataset) pair. PatchTST is generally strongest on MSE, while SRSNet is especially competitive on MAE. Nonetheless, the grid does not reproduce a clear dominance by SRSNet. The non-Transformers baselines are not dominated, as DLinear and TimeMixer remain competitive on several ETT columns, which is in line with prior work showing that simple linear models can be strong time-series baselines (Zeng et al., 2023).

Table 1: Multivariate forecasting results, averages over horizons $F \in \{96, 192, 336, 720\}$. **Red**: best. Blue: 2nd best. Dash: missing or empty released script. Full per-cell numbers in Appendix C.

Datasets	ETTh1		ETTh2		ETTm1		ETTm2	
Metrics	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
PatchTST	**0.521**	0.517	**0.306**	**0.378**	**0.400**	0.430	**0.206**	0.300
DLinear	–	–	0.313	0.389	0.406	**0.420**	0.208	0.303
iTransformer	–	–	0.320	0.384	0.420	0.438	0.223	0.314
TimesNet	–	–	0.341	0.395	0.527	0.494	0.238	0.319
TimeMixer	–	–	0.319	0.387	0.410	0.426	0.211	0.305
PatchMLP	0.551	0.533	–	–	–	–	0.220	0.311
SRSNet	0.528	**0.516**	0.314	0.379	0.404	0.426	0.206	**0.300**

Table 2: Plug-in study: MLP base (SRSNet w/o SRS) vs. SRSNet Full. Per-dataset averages across the four horizons. Improve% is the relative MSE drop (positive means +SRS is better).

Datasets	ETTh1		ETTh2		ETTm1		ETTm2	
Metrics	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
MLP base (= SRSNet w/o SRS)	0.528	0.514	0.306	0.375	0.409	0.426	0.203	0.296
+SRS (= SRSNet Full)	0.527	0.515	0.312	0.379	0.402	0.425	0.207	0.299
Improve (MSE)	+0.03%		−1.82%		+1.91%		−1.82%	

The signed gap between our SRSNet numbers and the paper's averages $|\Delta| \approx 18\%$ in absolute value. The signed differences are structured by dataset, with ETTh1 and ETTm1 achieving worse results than the paper, while ETTh2 and ETTm2 often appear better. We could not identify a single cause for the discrepancies, and possible contributing factors might include hardware, nondeterministic kernels, and small environment differences.

3.2 Plug-in study (paper Tab. 3)

In the plug-in comparison, we investigate the contribution of SRS on top of an MLP backbone. While the paper reports improvements in MSE and MAE in a range of 4–9%, the per-dataset average results of our reproduction reported in Table 2 show that adding SRS gives mixed, mostly small changes. SRS has lower measured MSE in 7 of 16 cells, but the majority of the differences are close to zero. We also did not find a clear long-horizon benefit; for example, at horizon 720, SRS outperforms the baseline in only one dataset (ETTm1), and that margin is negligible (−0.03%). The ETTm1 dataset is also the only one showing lower measured MSE at all time horizons.

3.3 Ablation (paper Tab. 4)

The component ranking in our ablation differs notably from the paper. The authors flagged Selective Patching as the largest individual contributor, while we identify that only removing Adaptive Fusion results in a clear increase of average MSE as per Table 3. Concretely, removing AF costs about +3.4% average MSE across the 16 ETT cells, whereas removing SRS as a whole, the Selective Patching alone, or the Dynamic Reassembly alone all stay within roughly 0.1% of the full model. Therefore, within our reproduction scope, AF appears to be the component carrying most of the measurable contribution of SRS and this is the strongest reproducible finding of our study, while SP and DR are not separable from a baseline that does not use them

Table 3: Ablation of key components in SRS, averages over horizons. **Red**: best. Blue: 2nd best. Per-cell numbers in Appendix D.

Datasets	ETTh1		ETTh2		ETTm1		ETTm2	
Metrics	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
w/o SRS	0.528	**0.514**	**0.306**	**0.375**	0.409	0.426	**0.203**	**0.296**
w/o Selective Patching	0.528	0.514	0.306	0.375	0.408	**0.425**	0.204	0.296
w/o Dynamic Reassembly	0.529	0.518	0.309	0.378	0.403	0.426	0.205	0.299
w/o Adaptive Fusion	0.551	0.538	0.323	0.390	0.406	0.430	0.216	0.310
SRSNet (Full)	**0.526**	0.515	0.313	0.379	**0.403**	0.426	0.205	0.299

Table 4: Extension results: averages over horizons. **Red**: best. Blue: 2nd best. Per-cell numbers with seed std in Appendix A.

Datasets	ETTh1		ETTh2		ETTm1		ETTm2	
Metrics	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
RandomSP	0.526	0.514	0.307	0.376	0.408	0.425	**0.204**	0.296
RandomSP + NoShuffle	**0.526**	**0.514**	0.305	0.374	0.408	0.425	0.204	**0.296**
TASP	0.527	0.515	**0.304**	**0.373**	0.407	0.426	0.207	0.299
HypernetAF	0.529	0.517	0.306	0.375	0.405	0.425	0.204	0.296
SRSNet (baseline)	0.528	0.516	0.312	0.379	**0.402**	**0.425**	0.205	0.298

4 Extension results

We trained the four single-axis variants of Section 2 (RandomSP, RandomSPNoShuffle, TASP, HypernetAF) and a SRSNet baseline on the full ETT grid (4 datasets × 4 horizons × 5 seeds). Each variant produces 80 paired observations against SRSNet. Before this full-grid run, we also ran a small 9-variant pilot with selection–fusion combinations on ETTh1 and ETTm2 at horizons 96 and 720; since the combinations did not show a clearer pattern, we kept them in Appendix B and focused the main text on single-axis variants.

Table 4 reports the per-dataset averages over horizons. The five variants land within 0.001–0.003 MSE of each other on every column: read at this resolution, replacing the learned scorer with random selection (RandomSP / RandomSPNoShuffle), using engineered features instead of raw patches (TASP), or making α a function of the batch context (HypernetAF) produces no visible difference from the baseline.

Aggregating the 80 paired deltas (Table 5) gives the same picture in summary form. The mean ΔMSE is at most 0.35% in magnitude for every variant; the seed-level standard deviation is $\sim 2\%$, an order of magnitude larger. The win counts are close to chance: 37–46 out of 80.

We are careful not to overstate this. These results are descriptive rather than a formal significance test: the average deltas are small, the win counts are mixed, and the differences are not clearly separated from seed-to-seed variation. What we can say is that, on this 400-run ETT grid, changing one component at a time did not show a systematic advantage over the SRSNet baseline.

4.1 Backbone extension (encoder)

We also tested whether the linear head can be replaced with something heavier. Three variants (SRSNet, TASP, HypernetAF) were re-run with a TransformerEncoder inserted between SRS and the FlattenHead, on the full ETT grid with 5 seeds each (240 additional runs). Table 6 reports the averages.

The encoder makes things worse by 2–4% MSE on average. The increase is mild on ETTh1, but more visible on ETTm2 long horizons. Among the three encoder variants, HypernetAF + Encoder is consistently the strongest, taking the second-best slot on every dataset in Table 6; vanilla SRSNet + Encoder and TASP + Encoder land slightly behind. Even so, none of the three encoder variants outperforms the plain SRSNet baseline. The paper's preference for a simple forecasting head is therefore consistent with our numbers: on

Table 5: Mean ΔMSE (%) versus SRSNet across 80 paired observations. "std %" is the seed-level standard deviation. "wins / 80" is how many paired observations the variant beats the baseline on; 40 is chance.

Variant	mean Δ% (std %)	wins / 80
RandomSP	−0.21 (1.93)	46
RandomSPNoShuffle	−0.34 (2.24)	45
TASP	−0.10 (2.07)	37
HypernetAF	−0.35 (1.89)	44

Table 6: Backbone extension: SRSNet + TransformerEncoder, on the full ETT grid. Per-cell numbers with seed std in Appendix E.

Datasets	ETTh1		ETTh2		ETTm1		ETTm2	
Metrics	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
SRSNet (baseline)	**0.528**	**0.516**	**0.312**	**0.379**	**0.402**	**0.425**	**0.205**	**0.298**
SRSNet + Encoder	0.537	0.532	0.334	0.392	0.403	0.431	0.218	0.312
TASP + Encoder	0.532	0.528	0.331	0.391	0.406	0.430	0.218	0.310
HypernetAF + Encoder	0.530	0.526	0.327	0.389	0.402	0.429	0.214	0.308

this grid, adding a Transformer encoder between SRS and the loss does not help, regardless of which selection or fusion variant we pair it with.

5 Discussion

Firstly, this report should be considered an ETT-focused reproduction under the constraints explained previously, and not a full re-evaluation of the original benchmarks. In general, SRSNet remains competitive with strong baselines, but our reproduced numbers differ noticeably from the paper.

Importantly, when we focus our attention on the component-level analysis, the results are more ambiguous than the headline accuracy. Even though the paper reports SP as the most important component, our ETT-only ablation instead presents the clearest degradation when AF is gone. Concurrently, taking away the learned selector with RandomSP (or RandomSPNoShuffle), as well as using TASP, does not introduce an obvious drop relative to SRSNet. This suggests that, in our settings, the evidence for learned patch selection as the main source of the reported gain is weak. On the other hand, this does not ultimately prove that learned SP is useless either; the measured effects with our extensions are altogether small, and seed variability is non-negligible. A larger study with more seeds and a pre-specified paired analysis over dataset-horizon cells, as well as the reproduction of the full benchmark, would be required to conclusively establish the existence of a smaller but real effect of the learned selector.

For this reproduction, the safest interpretation that we can offer is that SRSNet's ETT performance is reproducible as competitive, but not dominant, while the mechanistic explanation is less stable. AF appears to be key in our ablation, whereas learned selection does not obviously outperform engineered or even random alternatives. Lastly, the encoder extension also suggests that attaching extra capacity after SRS fails to meaningfully improve the results.

6 Conclusion

We reproduced the main ETT-focused SRSNet experiments and introduced our own added controls for selection, fusion, and the forecasting head (816 runs in total across reproduction and extensions). The headline comparison does not show a collapse of SRSNet on ETT, but it also does not reproduce the dominance over the other baselines as it was found in the paper. SRSNet stays close to the strongest ones,

chiefly on MAE, whereas PatchTST generally wins on MSE. Additionally, SP is not clearly separated from simpler selection controls, and AF is the most sensitive ablation in our runs.

Member contributions

All three authors contributed to the design of the experiments, the choice of variants to test, and the writing of the report. The code, the runner infrastructure, and the CSV / Markdown tables were a shared effort.

References

Taesung Kim, Jinhee Kim, Yunwon Tae, Cheonbok Park, Jang-Ho Choi, and Jaegul Choo. Reversible instance normalization for accurate time-series forecasting against distribution shift. In *International Conference on Learning Representations (ICLR)*, 2022.

Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *International Conference on Learning Representations (ICLR)*, 2023.

Xiangfei Qiu, Jilin Hu, Lekui Zhou, Xingjian Wu, Junyang Du, Buang Zhang, Chenjuan Guo, Aoying Zhou, Christian S. Jensen, Zhenli Sheng, and Bin Yang. Tfb: Towards comprehensive and fair benchmarking of time series forecasting methods. *Proceedings of the VLDB Endowment*, pp. 2363–2377, 2024.

Peiwang Tang and Weitai Zhang. Unlocking the power of patch: Patch-based mlp for long-term time series forecasting. In *AAAI Conference on Artificial Intelligence*, 2025.

Xingjian Wu, Xiangfei Qiu, Hanyin Cheng, Zhengyu Li, Jilin Hu, Chenjuan Guo, and Bin Yang. Enhancing time series forecasting through selective representation spaces: A patch perspective. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL `https://arxiv.org/abs/2510.14510`.

Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. Are transformers effective for time series forecasting? In *AAAI Conference on Artificial Intelligence*, 2023.

Yunhao Zhang and Junchi Yan. Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting. In *International Conference on Learning Representations (ICLR)*, 2023.

A Per-cell results on the full ETT grid (extensions)

Table 7 reports the per-cell mean MSE and seed-level standard deviation (5 seeds) for the five single-axis variants in the extension study, on the full ETT grid (4 datasets × 4 horizons = 16 cells). **Red**: best per row. Blue: 2nd best per row.

Table 7: Per-cell mean MSE ± seed-level standard deviation (5 seeds) for the five single-axis variants on the full ETT grid. **Red**: best per row. Blue: 2nd best per row.

Dataset	H	SRSNet	RandomSP	RandSP+NoSh	TASP	HypernetAF
ETTh1	96	0.4362±0.0004	0.4360±0.0004	0.4361±0.0002	0.4368±0.0005	**0.4360**±0.0004
ETTh1	192	0.4839±0.0026	0.4821±0.0004	**0.4818**±0.0006	0.4836±0.0003	0.4879±0.0045
ETTh1	336	0.5304±0.0035	0.5332±0.0050	0.5332±0.0050	0.5340±0.0066	**0.5303**±0.0026
ETTh1	720	0.6606±0.0026	0.6545±0.0033	0.6534±0.0015	**0.6533**±0.0044	0.6622±0.0020
ETTh2	96	0.2254±0.0015	0.2195±0.0014	**0.2173**±0.0002	0.2191±0.0032	0.2193±0.0006
ETTh2	192	0.2840±0.0039	0.2779±0.0024	**0.2756**±0.0021	0.2767±0.0032	0.2777±0.0047
ETTh2	336	0.3236±0.0060	0.3162±0.0021	**0.3129**±0.0002	0.3154±0.0023	0.3156±0.0016
ETTh2	720	0.4137±0.0115	0.4150±0.0071	0.4156±0.0090	**0.4047**±0.0120	0.4118±0.0152
ETTm1	96	**0.3171**±0.0026	0.3209±0.0004	0.3215±0.0004	0.3203±0.0017	0.3193±0.0007
ETTm1	192	**0.3687**±0.0013	0.3770±0.0005	0.3773±0.0005	0.3765±0.0016	0.3739±0.0014
ETTm1	336	**0.4177**±0.0058	0.4271±0.0010	0.4283±0.0020	0.4259±0.0029	0.4227±0.0025
ETTm1	720	**0.5037**±0.0017	0.5082±0.0043	0.5069±0.0013	0.5056±0.0046	0.5045±0.0059
ETTm2	96	0.1482±0.0008	0.1484±0.0004	0.1486±0.0004	0.1488±0.0011	**0.1476**±0.0004
ETTm2	192	0.1819±0.0017	0.1811±0.0010	0.1813±0.0007	0.1851±0.0017	**0.1804**±0.0008
ETTm2	336	0.2168±0.0026	0.2146±0.0003	**0.2146**±0.0005	0.2181±0.0029	0.2161±0.0023
ETTm2	720	0.2739±0.0023	**0.2706**±0.0014	0.2714±0.0033	0.2752±0.0043	0.2718±0.0037

B Small-grid 9-variant study (with combos)

Table 8 reports the per-cell mean MSE ± seed-level standard deviation for the small-grid pilot: 2 datasets (ETTh1, ETTm2) × 2 horizons $(96, 720)$ × 5 seeds with 9 variants (the same five single-axis variants of the main study, plus four multi-axis combinations: RandomSP × HypernetAF, TASP × HypernetAF, and their identity-shuffle counterparts). The combinations did not show a clearer pattern than the single-axis variants, which is why we dropped them from the main full-grid study.

Table 8: Per-cell mean MSE ± seed std (5 seeds) for the nine variants of the small-grid pilot. Abbreviations: Rand = RandomSP; RandNS = RandomSPNoShuffle; Hyp = HypernetAF; R+H = RandomSP + HypernetAF; R+NS+H = RandomSP + NoShuffle + HypernetAF; T+H = TASP + HypernetAF; T+NS+H = TASP + NoShuffle + HypernetAF. **Red**: best per row. Blue: 2nd best per row.

Ds	H	SRSNet	Rand	RandNS	TASP	Hyp	R+H	T+H	R+NS+H	T+NS+H
ETTh1	96	**0.4360**±.0006	0.4361±.0004	0.4361±.0002	0.4365±.0005	0.4361±.0004	0.4361±.0001	0.4371±.0006	0.4363±.0001	0.4365±.0004
ETTh1	720	0.6604±.0028	0.6545±.0034	**0.6534**±.0015	0.6538±.0039	0.6621±.0019	0.6546±.0036	0.6547±.0058	0.6545±.0031	0.6563±.0009
ETTm2	96	0.1481±.0011	0.1482±.0009	0.1486±.0004	0.1491±.0015	**0.1477**±.0002	0.1481±.0002	0.1483±.0003	0.1484±.0001	0.1485±.0002
ETTm2	720	0.2708±.0060	0.2706±.0014	0.2714±.0033	0.2739±.0034	**0.2697**±.0025	0.2707±.0035	0.2717±.0015	0.2697±.0026	0.2721±.0021

C Tab. 1 per-cell with MAE

Table 9 expands the headline comparison Table 1 of the main text with MAE values and shows the full per-cell numbers (16 cells per model).

D Tab. 3 (ablation) per-cell with MAE

Table 10 expands the ablation Table 3 of the main text with MAE values and shows the full per-cell numbers.

Table 9: Per-cell MSE and MAE on the four ETT datasets and four horizons, for SRSNet and the baselines we could re-run on Hivenet. Dash indicates empty or missing released scripts. **Red**: best per row per metric. Blue: 2nd best per row per metric.

Ds	H	PatchTST		DLinear		iTransf.		TimesNet		TimeMixer		PatchMLP		SRSNet	
		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	96	**0.419**	0.446	–	–	–	–	–	–	–	–	0.441	**0.446**	0.437	0.454
ETTh1	192	**0.461**	**0.479**	–	–	–	–	–	–	–	–	0.492	0.499	0.487	0.490
ETTh1	336	0.534	0.526	–	–	–	–	–	–	–	–	0.569	0.550	**0.529**	**0.517**
ETTh1	720	0.669	0.617	–	–	–	–	–	–	–	–	0.700	0.636	**0.658**	**0.603**
ETTh2	96	0.240	0.340	**0.216**	**0.315**	0.245	0.332	0.259	0.343	0.248	0.345	–	–	0.224	0.319
ETTh2	192	0.277	0.358	0.277	0.365	0.287	0.370	0.311	0.377	**0.276**	0.359	–	–	**0.276**	**0.358**
ETTh2	336	0.318	**0.387**	**0.312**	0.394	0.322	0.390	0.387	0.424	0.338	0.403	–	–	0.327	0.392
ETTh2	720	**0.388**	**0.428**	0.448	0.481	0.426	0.444	0.405	0.437	0.415	0.441	–	–	0.429	0.448
ETTm1	96	0.323	0.380	0.337	0.373	0.343	0.387	0.454	0.451	0.327	0.372	–	–	**0.320**	**0.371**
ETTm1	192	0.379	0.416	0.390	**0.405**	0.401	0.425	0.493	0.471	0.390	0.412	–	–	**0.371**	0.406
ETTm1	336	0.420	0.442	0.424	**0.433**	0.446	0.450	0.551	0.508	0.437	0.442	–	–	**0.419**	0.444
ETTm1	720	0.477	0.483	**0.473**	**0.471**	0.490	0.488	0.611	0.545	0.484	0.479	–	–	0.505	0.484
ETTm2	96	**0.147**	0.255	0.149	0.255	0.162	0.269	0.169	0.269	0.151	0.256	0.154	0.257	0.149	**0.253**
ETTm2	192	0.187	0.290	0.184	**0.284**	0.205	0.303	0.204	0.298	0.187	0.288	0.191	0.291	**0.183**	0.285
ETTm2	336	**0.214**	**0.306**	0.222	0.314	0.231	0.321	0.254	0.331	0.228	0.318	0.238	0.328	0.221	0.310
ETTm2	720	0.274	**0.349**	0.277	0.357	0.294	0.364	0.327	0.376	0.279	0.358	0.295	0.369	**0.272**	0.351

Table 10: Per-cell MSE and MAE for the ablation study, on the full ETT grid (16 cells per variant). **Red**: best per row per metric. Blue: 2nd best per row per metric.

Ds	H	w/o SRS		w/o SP		w/o DR		w/o AF		SRSNet	
		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	96	0.436	**0.452**	**0.436**	0.452	0.436	0.453	0.445	0.468	0.436	0.453
ETTh1	192	0.481	**0.483**	0.482	0.484	0.484	0.487	0.510	0.515	**0.481**	0.485
ETTh1	336	0.540	0.522	0.539	0.522	0.540	0.522	0.555	0.538	**0.529**	**0.518**
ETTh1	720	**0.653**	**0.598**	0.655	0.599	0.657	0.608	0.693	0.631	0.658	0.603
ETTh2	96	**0.217**	**0.315**	0.219	**0.315**	0.223	0.320	0.241	0.336	0.228	0.324
ETTh2	192	**0.276**	**0.357**	0.279	0.359	0.278	0.360	0.296	0.375	0.282	0.361
ETTh2	336	**0.313**	**0.381**	0.313	0.381	0.319	0.387	0.326	0.402	0.317	0.385
ETTh2	720	0.419	0.446	**0.414**	0.445	0.417	**0.444**	0.428	0.449	0.425	0.448
ETTm1	96	0.322	0.371	0.321	**0.370**	0.321	0.372	0.319	0.373	**0.317**	**0.370**
ETTm1	192	0.378	0.407	0.376	0.406	0.369	0.406	0.373	0.409	**0.369**	**0.405**
ETTm1	336	0.431	0.444	0.430	0.443	**0.414**	**0.438**	0.416	0.443	0.418	0.444
ETTm1	720	0.506	**0.480**	**0.504**	0.482	0.508	0.486	0.517	0.494	0.506	0.485
ETTm2	96	0.149	0.254	0.150	0.254	0.149	0.255	0.152	0.260	**0.148**	**0.253**
ETTm2	192	0.181	**0.280**	0.180	0.280	**0.180**	0.282	0.191	0.292	0.182	0.281
ETTm2	336	0.214	**0.305**	0.216	0.307	0.217	0.309	0.228	0.319	**0.214**	0.309
ETTm2	720	**0.268**	**0.344**	0.270	0.345	0.277	0.351	0.294	0.368	0.275	0.353

E Encoder-extension per-cell with MAE

Table 11 reports the per-cell mean MSE and MAE for the three encoder-extension variants (Table 6 of the main text).

F Implementation details

All extension layers are defined in `ts_benchmark/baselines/srs_paper/extensions.py` in our fork. They inherit from the original `SRS` class of Wu et al. (2025) and override exactly one method:

- `SRSRandomSP._select`: replaces `torch.argmax` on the scorer's output with a `torch.randint` over the candidate dimension.
- `SRSRandomSPNoShuffle._shuffle`: returns its input unchanged (identity shuffle).
- `SRSTimeAware._select`: computes per-window FFT magnitude, autocorrelation, variance, and trend slope; the scorer is a $4 \to 16 \to n$ MLP over those features.

Table 11: Per-cell mean MSE and MAE for the three encoder-extension variants (5 seeds per cell) on the full ETT grid. **Red**: best per row per metric. Blue: 2nd best per row per metric.

Ds	H	Enc		TASP+Enc		Hyp+Enc	
		MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	96	0.433	0.462	0.436	0.464	**0.431**	**0.461**
ETTh1	192	0.491	0.503	0.492	0.503	**0.487**	**0.499**
ETTh1	336	**0.532**	**0.526**	0.532	0.527	0.535	0.528
ETTh1	720	0.691	0.635	0.668	**0.617**	**0.667**	0.618
ETTh2	96	0.242	0.335	0.244	0.335	**0.238**	**0.333**
ETTh2	192	0.299	0.374	0.299	0.375	**0.297**	**0.372**
ETTh2	336	**0.343**	**0.398**	0.345	0.402	0.347	0.402
ETTh2	720	0.451	0.463	0.434	0.454	**0.426**	**0.448**
ETTm1	96	0.328	0.377	**0.326**	**0.375**	0.330	0.378
ETTm1	192	0.390	0.420	0.387	0.415	**0.376**	**0.410**
ETTm1	336	0.421	0.444	0.426	0.445	**0.421**	**0.444**
ETTm1	720	**0.475**	**0.482**	0.487	0.486	0.483	0.485
ETTm2	96	0.153	0.262	0.153	0.258	**0.152**	**0.257**
ETTm2	192	0.195	0.296	0.199	0.297	**0.189**	**0.293**
ETTm2	336	0.231	0.323	**0.228**	**0.320**	0.228	0.320
ETTm2	720	0.294	0.369	0.292	0.365	**0.287**	**0.360**

- `SRSHypernetAF.forward`: zero-initialised hypernet plus bias = 3.5 produces the fusion weights from a batch-level mean of the original-view embedding.

The encoder-extension wrappers (`SRSNet_TransformerEncoder`, etc.) wrap the SRS module in a TransformerEncoder backbone with two layers, four heads, and feedforward dimension $4 \times d_{\text{model}}$ before the FlattenHead. They are defined in the same file as the other extensions and share the same training pipeline.

The runner exposes the variants through three scopes in `scripts/repro/paper_repro.py`: `selectivity-controls` (small grid, 9 variants), `selectivity-controls-full-simple` (full grid, 5 single-axis variants), and `encoder-extension` (full grid, 3 encoder variants). Because the runner skips already- completed metadata files, every scope is fully resume-aware.

G Hyper-parameters

Variants inherit the SRSNet hyper-parameters from the released dataset/horizon scripts. We only apply the paper-faithful runner overrides `batch_size=64`, `train_drop_last=false`, `num_workers=0`, and the model-name/seed changes needed for each variant.